

WEB DEVELOPMENTFRAMEWORK

Reading Material

Unit 2 — Cascading Style Sheets

Unit 3 — JavaScript Fundamentals

Unit 2 — Cascading Style Sheets (CSS)

HTML gives a page its **structure** — headings, paragraphs, lists, forms. But a browser's built-in styling is plain: black text on a white background, Times New Roman, everything stacked top to bottom. CSS (Cascading Style Sheets) is the **presentation layer**. It decides color, spacing, typography, layout and position, without touching the meaning of the content.



Figure 2.1 — The three layers of a web page. Keeping them separate is the single most important habit in front-end development.

The word **Cascading** is the key idea in the name. Many rules can target the same element at the same time. The cascade is the set of tie-breaking rules the browser uses to decide which declaration finally wins. Almost every confusing CSS bug a beginner hits — "why is my text still blue?" — is really a cascade question.

2.1 Inline, Internal and External CSS

There are exactly three places CSS can live. They differ in **where the code sits**, not in what CSS can do.

(a) Inline CSS — the style attribute

The declarations are written directly on one HTML element using the **style** attribute. There is no selector, because the element the rule applies to is the element it is written on.

```
<p style="color: #C00000; font-size: 18px; text-align: center;">  
  Result declaration will be announced on 25 August.  
</p>
```

Inline CSS affects exactly one element and nothing else.

- **Advantage:** quick, and useful when a value is generated dynamically by JavaScript or a server (for example a progress bar width).
- **Disadvantage:** it cannot be reused, it mixes presentation into structure, it cannot express **:hover** or media queries, and it makes the HTML long and hard to read.
- **Verdict:** avoid it in coursework except for genuinely one-off, dynamic values.

(b) Internal CSS — the <style> element

The CSS is written once inside a **<style>** element, normally placed in the **<head>**. It applies to that **one HTML document only**.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>StudentHub</title>
  <style>
    body { font-family: Arial, sans-serif; background: #F4F6F8; }
    .card { background: #FFFFFF; padding: 16px; border-radius: 8px; }
    .card h2 { color: #1F4E79; margin-top: 0; }
  </style>
</head>
<body>
  <div class="card"><h2>Upcoming Events</h2><p>Tech Fest 2026</p></div>
</body>
</html>

```

Internal CSS: one `<style>` block styles every matching element in this file.

- **Advantage:** no extra file to manage; good for a single demo page, a prototype, or an emailed HTML file.
- **Disadvantage:** the styles cannot be shared with a second page, so a ten-page site means the same CSS copied ten times.
- **Verdict:** acceptable for a one-page exercise; not for a real site.

(c) External CSS — a separate .css file

The CSS is written in its own file (say `style.css`) with **no HTML tags inside it at all**, and every page links to it from the `<head>` using the `<link>` element.

```

<!-- index.html -->
<head>
  <link rel="stylesheet" href="css/style.css">
</head>

```

```

/* css/style.css — notice: no <style> tag inside a .css file */
body { font-family: Arial, sans-serif; background: #F4F6F8; }
.card { background: #FFF; padding: 16px; border-radius: 8px; }

```

One stylesheet, linked from every page of the site.

- **Advantage:** written once, used by every page. Change the brand colour in one line and all fifty pages update. The browser also **caches** the file, so pages after the first one load faster.
- **Disadvantage:** one extra HTTP request (negligible in practice).
- **Verdict:** this is the professional default. Use it for all project work.

Common exam question — "Which method should be preferred and why?" Answer: **external**, because of reusability, maintainability, caching, and clean separation of content from presentation.

Comparison at a glance

Point	Inline	Internal	External
Written in	style attribute on the tag	<code><style></code> in <code><head></code>	separate .css file

Point	Inline	Internal	External
Scope	one element	one page	whole website
Reusable	No	Within that page only	Yes
Selectors allowed	No	Yes	Yes
Media queries / :hover	No	Yes	Yes
Browser caching	No	No	Yes
Cascade priority	Highest	Depends on order	Depends on order
Best used for	dynamic one-off values	single demo page	real projects

Which one wins when they conflict?

If the same property is set on the same element from more than one place, the browser resolves it in this order:

1. A declaration marked **!important** beats everything else (use it sparingly).
2. **Inline** style beats internal and external, because it is the most specific place possible.
3. Between internal and external, **higher specificity wins** (see section 2.4).
4. If specificity is equal, **whichever is written last wins**. So a `<style>` block placed *after* the `<link>` overrides it, and a `<link>` placed after the `<style>` overrides that.

```
p { color: green; }      /* external file */
p { color: blue; }      /* internal <style>, written later -> blue wins */

<p style="color: red;"> /* inline -> red wins over both */
```

Same selector, same specificity: the later rule wins. Inline beats both.

2.2 CSS Syntax and Comments

A CSS file is simply a list of **rule sets**. Every rule set has the same shape:

```
selector {
  property: value;      <-- one declaration
  property: value;      <-- another declaration
}

h1 {
  color: #1F4E79;
  font-size: 32px;
  text-align: center;
}
```

Term	In the example	Meaning
Selector	h1	which elements the rule targets
Declaration block	everything inside { }	the group of declarations
Declaration	color: #1F4E79;	one property/value pair

Term	In the example	Meaning
Property	color	the aspect being changed
Value	#1F4E79	what it is being changed to
Separator	:	colon between property and value
Terminator	;	semicolon after each declaration

The semicolon after the **last** declaration is technically optional, but always type it — the day you add another line below it, forgetting the semicolon silently breaks both declarations.

Comments

CSS has exactly one comment form: `/* ... */`. It can span multiple lines. There is **no** `//` line comment in CSS — writing `// hello` will break the rule that follows it.

```
/* ===== Header styles ===== */
.navbar {
  background: #1F4E79; /* brand navy */
  /* height: 80px; temporarily disabled for testing */
}

/* WRONG — CSS has no line comments
// .navbar { color: red; }
*/
```

Comments are used for three things: labelling sections of a long stylesheet, explaining a value that is not obvious (a magic number, a browser workaround), and temporarily disabling a rule while debugging.

Whitespace and case. CSS ignores extra spaces, tabs and blank lines, so indent freely for readability. Property names and most keyword values are case-insensitive (COLOR: RED works), but **class and id names in selectors are case-sensitive** — `.Card` and `.card` are two different classes.

2.3 Selectors

A selector is a pattern that answers the question "which elements should this rule apply to?". Mastering selectors is most of learning CSS.

The core selectors

Type	Syntax	Selects	Example
Universal	*	every element	<code>* { margin: 0; }</code>
Element (type)	p	every <p> on the page	<code>p { line-height: 1.6; }</code>
Class	.name	every element with class="name"	<code>.card { padding: 16px; }</code>
ID	#name	the one element with id="name"	<code>#main-nav { position: sticky; }</code>
Grouping	a, b, c	all of a, b and c	<code>h1, h2, h3 { color: navy; }</code>

Type	Syntax	Selects	Example
Descendant	a b	every b anywhere inside a	nav a { color: white; }
Child	a > b	b that is a direct child of a	ul > li { list-style: none; }
Attribute	[attr]	elements having that attribute	input[required] { ... }
Pseudo-class	a:state	elements in a particular state	a:hover { color: red; }
Pseudo-element	a::part	a sub-part of an element	p::first-letter { ... }

Class vs ID — the distinction examiners look for

	Class	ID
Symbol in CSS	.classname	#idname
Reuse on a page	Any number of elements	Must be unique — one element only
Elements per rule	Many	One
Multiple on one element	Yes: class="card featured"	No — one id per element
Specificity weight	10	100
Typical use	Repeated components: buttons, cards	Unique landmarks, JS hooks, #anchor links

```
<div class="card featured" id="event-2026">...</div>

.card      { background: #fff; border-radius: 8px; } /* many cards */
.featured { border: 2px solid gold; }             /* only some */
#event-2026 { order: -1; }                        /* this one */
```

An element may carry several classes separated by spaces, but only one id.

Grouping

When several selectors need identical declarations, list them separated by commas instead of repeating the block. A common beginner error is using a space instead of a comma — **h1 h2** means "an h2 inside an h1", which is not what you meant.

```
h1, h2, h3 { font-family: Georgia, serif; color: #1F4E79; }

/* equivalent to writing three separate rule sets */
```

Descendant vs child — read this carefully

Both combine two selectors, but they differ in how deep they look. The **descendant** combinator (a space) matches at any depth. The **child** combinator (>) matches only one level down.

```

<div class="menu">
  <a href="#">Home</a>                                <!-- direct child, depth 1 -->
  <ul>
    <li><a href="#">Events</a></li> <!-- grandchild, depth 3 -->
  </ul>
</div>

.menu a { color: red; } /* matches BOTH links */
.menu > a { color: blue; } /* matches only "Home" */

```

Combinator	Symbol	Meaning	Example
Descendant	space	anywhere inside	article p
Child	>	immediate child only	article > p
Adjacent sibling	+	the very next sibling	h2 + p
General sibling	~	any later sibling	h2 ~ p

Attribute selectors

These match on the presence or the value of an HTML attribute — extremely useful for styling form controls and links without adding extra classes.

Selector	Matches	Typical use
[disabled]	attribute is present, any value	input[disabled] { opacity: .5; }
[type="email"]	value is exactly "email"	styling one kind of input
[href^="https"]	value starts with (^)	marking secure links
[href\$=".pdf"]	value ends with (\$)	adding a PDF icon
[class*="btn"]	value contains (*)	matching btn-primary, btn-lg
[lang ="en"]	value is en or begins en-	language variants
[title~="lab"]	one space-separated word is lab	word matching in a list

```

input[type="text"], input[type="email"] {
  border: 1px solid #B4C6D9;
  padding: 10px;
}
a[href$=".pdf"]::after { content: " (PDF)"; font-size: 0.85em; }
a[target="_blank"]::after { content: " \2197"; } /* north-east arrow */

```

Pseudo-classes — styling a state

A pseudo-class (single colon) selects elements that are in a particular **state** or at a particular **position** among siblings. Nothing extra is added to the HTML.

Pseudo-class	Applies when	Example
:hover	pointer is over the element	button:hover { background: #2E74B5; }

Pseudo-class	Applies when	Example
:active	element is being clicked	button:active { transform: scale(.98); }
:focus	element has keyboard focus	input:focus { outline: 2px solid #2E74B5; }
:focus-visible	focused via keyboard, not mouse	preferred for accessible outlines
:visited	link already visited	a:visited { color: purple; }
:checked	checkbox/radio is ticked	input:checked + label { font-weight: 700; }
:disabled	form control is disabled	input:disabled { background: #eee; }
:first-child	first child of its parent	li:first-child { border-top: none; }
:last-child	last child of its parent	li:last-child { border-bottom: none; }
:nth-child(n)	the nth child; also odd/even/2n+1	tr:nth-child(even) { background: #f5f5f5; }
:not(sel)	everything except sel	li:not(.active) { opacity: .7; }
:root	the <html> element	used to declare CSS variables

Link order matters. The link pseudo-classes must be written in the order **L-V-H-A** — :link, :visited, :hover, :active. Remember it as "LoVe Hate". Written out of order, a later rule of equal specificity cancels the earlier one and hover appears not to work.

```
tr:nth-child(even) { background: #F2F6FA; } /* zebra-striped table */
li:nth-child(3)   { color: crimson; }     /* exactly the 3rd      */
li:nth-child(3n)  { font-weight: bold; }  /* 3rd, 6th, 9th ...   */
.card:not(.featured) { border: 1px solid #ddd; }
```

Pseudo-elements — styling a part of an element

A pseudo-element (double colon ::) styles a **portion** of an element, or inserts generated content that does not exist in the HTML at all.

Pseudo-element	Styles	Note
::before	generated box before the content	requires the content property
::after	generated box after the content	requires the content property
::first-letter	the first letter of a block	for drop-cap effects
::first-line	the first rendered line	changes with window width
::selection	text the user has highlighted	only a few properties allowed
::placeholder	placeholder text in an input	colour, style, opacity
::marker	the bullet or number of a list item	colour and size of the marker

```
.required-field label::after {
  content: " *";          /* mandatory, may be "" for pure decoration */
  color: #C00000;
}

.article p::first-letter {
  font-size: 3em; float: left; line-height: 1; padding-right: 6px;
}

input::placeholder { color: #9AA5B1; font-style: italic; }
```

Single or double colon? Modern CSS uses `::` for pseudo-**elements** and `:` for pseudo-**classes**. Browsers still accept the old single colon on `:before/after` for backwards compatibility, but write `::` in new code. Generated content from `::before/::after` is decorative — screen readers may skip it, so never put essential information there.

2.4 Colour Theory, Colour Contrast and Font Properties

Ways to write a colour

Notation	Example	Explanation
Named colour	red, tomato, navy	147 predefined names; easy but imprecise
Hexadecimal	#1F4E79	#RRGGBB, each pair 00–FF (0–255)
Short hex	#F60	expands to #FF6600; only for repeated digits
8-digit hex	#1F4E7980	last pair is alpha (80 ≈ 50% opacity)
rgb()	rgb(31, 78, 121)	red, green, blue each 0–255
rgba()	rgba(31, 78, 121, 0.5)	same plus alpha 0 (clear) to 1 (solid)
hsl()	hsl(207, 59%, 30%)	hue 0–360°, saturation %, lightness %
hsla()	hsla(207, 59%, 30%, .8)	hsl plus alpha
transparent	transparent	fully see-through
currentColor	currentColor	reuses the element's own color value

HSL is the easiest model to reason about. Keep the hue fixed and change only the lightness to generate a whole tint/shade family for one brand colour — `hsl(207, 59%, 20%)`, `hsl(207, 59%, 30%)`, `hsl(207, 59%, 45%)` are guaranteed to look related. Doing the same with hex requires guesswork.

Basic colour theory for a web page

- **Hue** is the colour itself (red, blue). **Saturation** is its intensity. **Lightness/value** is how close it is to white or black.
- **Complementary** colours sit opposite on the colour wheel (blue/orange, red/green). High energy — ideal for a call-to-action button against the page background.

- **Analogous** colours sit next to each other (blue, blue-green, green). Calm and harmonious — good for backgrounds and large areas.
- **Triadic** colours are three evenly spaced hues. Vibrant but needs one dominant colour and two accents.
- **Monochromatic** schemes use one hue at several lightness levels. Safest choice for a student project — it is almost impossible to make it look wrong.
- **Warm** colours (red, orange, yellow) advance and feel energetic; **cool** colours (blue, green, violet) recede and feel calm and professional — which is why institutional sites are overwhelmingly blue.

The 60-30-10 rule. Give roughly 60% of the visible area to a neutral/dominant colour (usually a near-white background), 30% to a secondary colour (headers, panels), and 10% to an accent used only for the things you want clicked. This single rule fixes most amateur-looking colour schemes.

Font properties

Property	Common values	Notes
font-family	"Segoe UI", Arial, sans-serif	a fallback stack, ending in a generic family
font-size	16px, 1.25rem, 1.2em, 120%	rem = relative to root; em = relative to parent
font-weight	normal, bold, 100–900	400 = normal, 700 = bold
font-style	normal, italic, oblique	italic for emphasis
font-variant	normal, small-caps	capitals at lowercase height
line-height	1.5, 24px, 150%	unitless value is best — it scales with font-size
letter-spacing	normal, 0.5px, -0.02em	tracking; slight negative works on big headings
word-spacing	normal, 2px	space between words
text-align	left, right, center, justify	horizontal alignment of inline content
text-decoration	none, underline, line-through	used to remove link underlines
text-transform	none, uppercase, lowercase, capitalize	changes display only, not the source text
text-indent	30px, 2em	indents the first line of a paragraph
text-shadow	1px 1px 3px rgba(0,0,0,.4)	x, y, blur, colour

```
body {
  font-family: "Segoe UI", Roboto, Arial, sans-serif;
  font-size: 16px;
  line-height: 1.6;
  color: #1A1A1A;
}
h1 { font-size: 2.25rem; font-weight: 700; letter-spacing: -0.5px; }

/* shorthand: style variant weight size/line-height family */
p { font: italic normal 400 16px/1.6 Georgia, serif; }
```

In the font shorthand, size and family are compulsory and must come last, in that order.

The five **generic families** are **serif**, **sans-serif**, **monospace**, **cursive** and **fantasy**. Always end a font stack with one of them so the browser has a sensible last resort. Web-safe fonts (Arial, Georgia, Times New Roman, Courier New, Verdana, Tahoma) exist on nearly every device; anything else should be loaded as a web font:

```
<!-- in <head>, before your own stylesheet -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="stylesheet"
      href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600">

/* then in CSS */
body { font-family: "Poppins", Arial, sans-serif; }
```

px vs em vs rem. px is fixed. em multiplies the **parent's** font-size, so nested elements compound (1.2em inside 1.2em = 1.44x) — a classic source of runaway text. rem multiplies the **root** font-size (16px by default), so it is predictable and still respects a user who enlarges text in browser settings. Prefer rem for font sizes and em for padding that should scale with its own text.

2.5 The Box Model

Every element rendered by a browser is a rectangular box — even a single word inside a `<span>`. That box is built from four concentric layers. Understanding them is the difference between fighting CSS layout and controlling it.

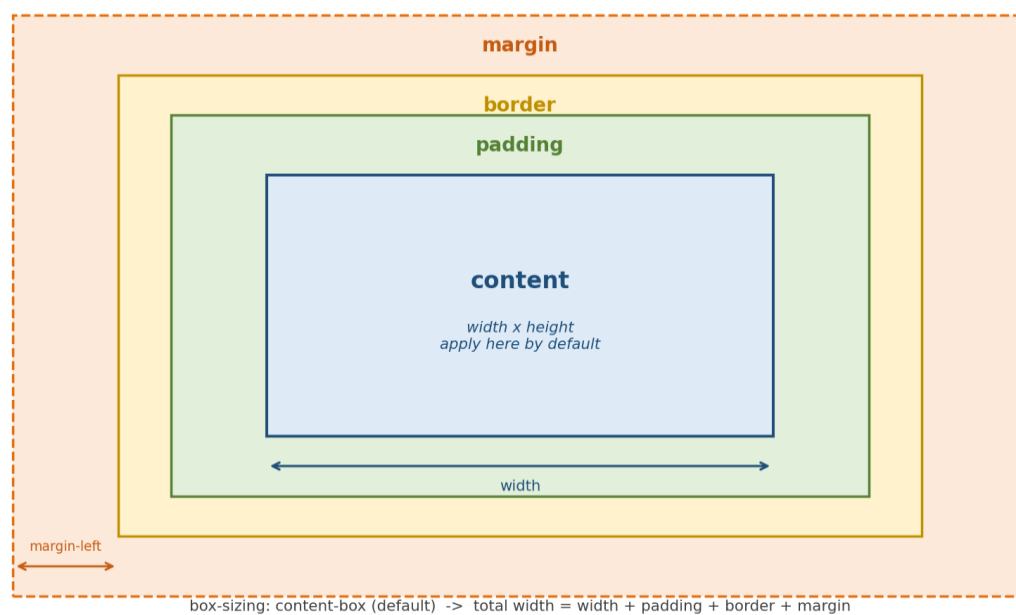


Figure 2.2 — The CSS box model.

Layer	What it is	Can it have a background?
Content	the text, image or child boxes; sized by width/height	yes

Layer	What it is	Can it have a background?
Padding	clear space inside the border, around the content	yes — it shows the element's background
Border	the line drawn around the padding	it is the line itself
Margin	clear space outside the border, separating neighbours	no — always transparent

width, height and the total size calculation

By default **width** and **height** size the **content box only**. Padding and border are added on top, which surprises nearly every beginner.

```
.box {
  width: 300px;
  padding: 20px;
  border: 5px solid #1F4E79;
  margin: 10px;
}
```

The rendered width is **not** 300px:

- Content = 300px
- plus left padding 20 + right padding 20 = 340px
- plus left border 5 + right border 5 = **350px of visible box**
- plus left margin 10 + right margin 10 = **370px of space occupied on the page**

This is why a layout of four boxes at **width: 25%** with padding overflows its row and wraps.

box-sizing — the fix

Value	width means	Effect
content-box (default)	width of the content only	padding and border enlarge the box
border-box	width of content + padding + border	padding and border shrink the content inward

```
/* Put this at the top of every stylesheet you write. */
*, ::before, ::after {
  box-sizing: border-box;
}

.box { width: 300px; padding: 20px; border: 5px solid #1F4E79; }
/* Now the box really is 300px wide; content shrinks to 250px. */
```

The universal border-box reset — standard practice in professional stylesheets.

Shorthand values: the clock rule

margin and **padding** accept one to four values, applied **clockwise starting from the top**: Top, Right, Bottom, Left (remember **TRouBLE**).

Written as	Top	Right	Bottom	Left
padding: 20px;	20	20	20	20
padding: 10px 20px;	10	20	10	20
padding: 10px 20px 30px;	10	20	30	20
padding: 10px 20px 30px 40px;	10	20	30	40

Individual sides can also be set directly: `margin-top`, `padding-left`, `border-bottom`, and so on. A later individual property overrides the earlier shorthand.

Two behaviours worth knowing

- **Centring with auto.** `margin: 0 auto;` on a block element that has an explicit `width` splits the leftover horizontal space equally, centring it in its parent. It does nothing without a width, and does not centre vertically.
- **Margin collapse.** Vertical margins between two adjacent block elements do not add up — the larger one is used. A 30px bottom margin above a 20px top margin produces a 30px gap, not 50px. Horizontal margins never collapse, and padding never collapses.
- **min-width / max-width / min-height / max-height** constrain a box: `max-width: 1100px; margin: 0 auto;` is the standard page-container recipe, giving a fluid layout that stops growing on large screens.
- Percentage `width` is relative to the **parent's** width; percentage `padding` and `margin` are **also** relative to the parent's **width**, even on the top and bottom.

2.6 Page Formatting

background

Property	Values	Purpose
background-color	any colour value	flat fill behind content and padding
background-image	<code>url("img/hero.jpg")</code> , <code>linear-gradient(...)</code>	image or gradient
background-repeat	<code>repeat</code> , <code>no-repeat</code> , <code>repeat-x</code> , <code>repeat-y</code>	tiling behaviour
background-position	<code>center</code> , <code>top left</code> , <code>50% 20%</code> , <code>20px 40px</code>	where the image starts
background-size	<code>auto</code> , <code>cover</code> , <code>contain</code> , <code>200px 100px</code>	<code>cover</code> fills and crops; <code>contain</code> fits fully
background-attachment	<code>scroll</code> , <code>fixed</code> , <code>local</code>	<code>fixed</code> gives a parallax-like effect
background (shorthand)	<code>color image repeat position/size attachment</code>	all of the above in one line

```

.hero {
  background-image: url("../img/campus.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  min-height: 60vh;
}

/* shorthand version */
.hero { background: #1F4E79 url("../img/campus.jpg") no-repeat center/cover; }

/* gradient – no image file needed */
.banner { background: linear-gradient(135deg, #1F4E79, #2E74B5); }

/* dark overlay so white text stays readable over a photo */
.hero {
  background:
    linear-gradient(rgba(0,0,0,.55), rgba(0,0,0,.55)),
    url("../img/campus.jpg") center/cover no-repeat;
}

```

Layered backgrounds are listed front to back, separated by commas.

border

A border needs three things: a **width**, a **style** and a **colour**. If the style is missing nothing is drawn, because the initial style is **none** — this is the single most common "my border isn't showing" bug.

```

border-width: 2px;
border-style: solid; /* solid dashed dotted double
                    groove ridge inset outset none */
border-color: #1F4E79;

/* shorthand – width style color */
border: 2px solid #1F4E79;

/* one side only */
border-bottom: 3px solid #2E74B5;
border-left: 4px solid crimson; /* the "quote bar" pattern */

/* removing a border */
border: none; /* or border: 0; */

```

border-radius

```

.card { border-radius: 8px; } /* all four corners */
.tag { border-radius: 999px; } /* pill shape */
.avatar { width: 80px; height: 80px; border-radius: 50%; } /* circle */
.tab { border-radius: 8px 8px 0 0; } /* TL TR BR BL */
.leaf { border-radius: 50% 0 50% 0; }
.blob { border-radius: 60% 40% 30% 70% / 60% 30% 70% 40%; } /* elliptical */

```

A perfect circle needs equal width and height plus border-radius: 50%.

box-shadow and text-shadow

box-shadow: offset-x offset-y blur spread color; — the first two are required, the rest optional. Add the keyword **inset** to draw the shadow inside the box.

```

.card      { box-shadow: 0 2px 6px rgba(0,0,0,0.12); } /* subtle lift */
.card:hover { box-shadow: 0 8px 20px rgba(0,0,0,0.18); } /* raised */
.well      { box-shadow: inset 0 2px 4px rgba(0,0,0,0.15); } /* pressed in */

/* multiple shadows, comma separated – first one is on top */
.fancy { box-shadow: 0 1px 2px rgba(0,0,0,.08), 0 8px 24px rgba(0,0,0,.12); }

/* text-shadow: x y blur color */
.hero h1 { text-shadow: 0 2px 6px rgba(0,0,0,0.6); }

```

Design tip. Real shadows fall downwards and are soft. A believable card shadow has a small positive y-offset, a generous blur and a **low-opacity black** — not a solid grey. `0 2px 6px rgba(0,0,0,.12)` looks professional; `5px 5px 0 grey` looks like 1998.

display

display decides what kind of box an element generates and how its children are laid out. It is the most powerful property in CSS.

Value	Starts new line?	Respects width/height?	Vertical margin/padding?
block (div, p, h1, section)	Yes — full width	Yes	Yes
inline (span, a, strong, em)	No — flows in text	No (ignored)	Padding shows but does not push neighbours
inline-block (button, input, img)	No	Yes	Yes
none	Element is removed entirely	—	—
flex	Block-level; children become flex items in a row/column	Yes	Yes
grid	Block-level; children are placed on a two-dimensional grid	Yes	Yes
inline-flex / inline-grid	Same as above but the container itself sits inline	Yes	Yes

```

nav ul { display: flex; gap: 24px; list-style: none; }
nav li { /* now a flex item, sits in a row automatically */ }

.badge { display: inline-block; padding: 4px 10px; border-radius: 999px; }
/* inline-block is needed: padding on a plain inline element would not
   push the surrounding lines apart. */

.mobile-only { display: none; } /* removed from the page completely */

```

visibility, and how it differs from display: none

Declaration	Visible?	Still takes up space?	Clickable?	Animatable?
display: none	No	No — layout closes up	No	No
visibility: hidden	No	Yes — the gap remains	No	No
opacity: 0	No	Yes	Yes — still clickable	Yes

visibility: collapse is a special value for table rows and columns: it removes the row without changing the table's overall layout. Elsewhere it behaves like **hidden**.

Accessibility note. **display: none** and **visibility: hidden** also hide content from **screen readers**. To hide something visually while keeping it available to assistive technology, use a **.visually-hidden** utility class (clip the element to a 1x1 pixel) rather than these properties.

overflow

overflow controls what happens when the content is bigger than the box that holds it.

Value	Behaviour
visible (default)	content spills outside the box and is still drawn
hidden	the overflowing part is clipped and cannot be reached
scroll	clipped, and scrollbars are always shown (even if not needed)
auto	clipped, and scrollbars appear only when needed — usually the right choice
clip	like hidden, but disallows any programmatic scrolling

```
.table-wrap { overflow-x: auto; }          /* wide table scrolls on mobile */
.chat-log { height: 320px; overflow-y: auto; }

/* Truncate one line of text with an ellipsis (all three lines needed) */
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

2.7 Positioning

By default every element sits in the **normal document flow**: blocks stack downwards, inline content flows left to right. The **position** property lets you take an element out of that flow or nudge it around within it. The offset properties **top**, **right**, **bottom** and **left** only have an effect when **position** is something other than **static**.

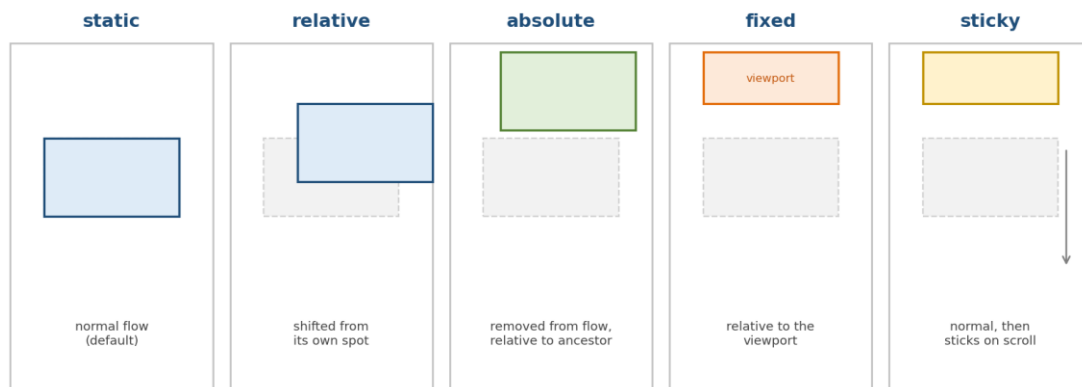


Figure 2.3 — The five position values. The dashed rectangle marks the space the element would have occupied in normal flow.

Value	In the flow?	Offsets measured from	Typical use
static	Yes (default)	n/a — offsets ignored	ordinary content
relative	Yes — original space is kept	its own normal position	small nudges; creating a positioning context
absolute	No — removed from flow	nearest positioned ancestor, else the page	badges, tooltips, close buttons, overlays
fixed	No — removed from flow	the viewport (browser window)	sticky headers, back-to-top button, modals
sticky	Yes, until the threshold is crossed	nearest scrolling ancestor	section headings, table headers, sidebars

relative + absolute: the pattern you will use most

position: absolute positions an element against its nearest ancestor whose position is **not** static. If there is none, it uses the initial containing block — effectively the page — and the element often flies to the top-left corner. The fix is to mark the intended parent **position: relative**.

```
<div class="event-card">
  <span class="badge">NEW</span>
  <h3>Tech Fest 2026</h3>
</div>
```

```

.event-card {
  position: relative;      /* <-- creates the reference frame */
  padding: 20px;
  border: 1px solid #D6DCE4;
  border-radius: 8px;
}

.badge {
  position: absolute;      /* positioned against .event-card */
  top: -10px;
  right: 12px;
  background: #C00000;
  color: #fff;
  padding: 3px 10px;
  border-radius: 999px;
  font-size: 12px;
}

```

Remove position: relative from the card and the badge jumps to the top-right of the whole page.

fixed and sticky

```

/* Navbar glued to the top of the window at all times */
.site-header {
  position: fixed; top: 0; left: 0; width: 100%;
  background: #1F4E79; z-index: 100;
}
body { padding-top: 64px; } /* fixed elements leave no gap – add it back */

/* Section heading that scrolls normally, then pins 0px from the top */
.section-title { position: sticky; top: 0; background: #fff; }

```

Sticky not working? Three requirements: (1) you must specify a threshold — top, bottom, left or right; (2) the element's parent must be taller than the element, because sticky only travels within its parent; (3) no ancestor may have overflow: hidden or auto, which silently disables it.

z-index and stacking

When boxes overlap, **z-index** decides which is drawn on top. A larger number is nearer the viewer. Two rules matter:

- **z-index** only works on **positioned** elements (relative, absolute, fixed, sticky) — and on flex/grid items. On a static element it is ignored.
- It is compared **within a stacking context**, not globally. A positioned element with a z-index creates a new context, and its children are stacked inside it. A child with **z-index: 9999** can never escape a parent whose context sits below another — this is why an enormous z-index sometimes changes nothing.

```
.modal-overlay { position: fixed; inset: 0;
                background: rgba(0,0,0,.5); z-index: 900; }
.modal-box     { position: relative; z-index: 901; }
.dropdown      { position: absolute; z-index: 50; }

/* Negative values push behind the parent background */
.watermark     { position: absolute; z-index: -1; }
```

Keep a small documented scale (e.g. dropdown 50, header 100, modal 900) rather than escalating to 99999.

float and clear

float was designed for one job: letting text wrap around an image, as in a printed magazine. Before Flexbox and Grid existed it was also abused to build whole page layouts — you will meet that in older code, but you should not write new layouts with it.

```
.article img {
  float: left;          /* left | right | none */
  margin: 0 16px 8px 0;
}
/* Text in the following paragraphs now wraps around the image. */
```

A floated element is partly removed from the flow, so its parent may **collapse to zero height**. **clear** tells an element to move below any floats:

Declaration	Effect
clear: left	moves below any left-floated elements
clear: right	moves below any right-floated elements
clear: both	moves below all floats — the usual choice
clear: none	default; no clearing

```
/* Classic clearfix — forces a parent to contain its floats */
.clearfix::after { content: ""; display: block; clear: both; }

/* Modern one-line replacement */
.container { display: flow-root; }
```

What to use today. For page layout use **Flexbox** (one dimension — a navbar, a row of cards) or **CSS Grid** (two dimensions — a full page skeleton). Keep **float** for its original purpose, wrapping text around a figure.

2.8 Unit 2 — Quick Revision

Question	One-line answer
What does the C in CSS stand for?	Cascading — the rule system that resolves conflicting declarations
Three ways to apply CSS?	Inline (style attribute), internal (<style>), external (<link> to .css)
Which is preferred and why?	External — reusable, cacheable, maintainable, separates concerns
CSS comment syntax?	/* comment */ only — there is no // in CSS
Difference between class and id?	Class is reusable and worth 10; id is unique and worth 100
div p vs div > p?	Descendant (any depth) vs direct child (one level)
Order of link pseudo-classes?	:link, :visited, :hover, :active — LoVe HAtE
Pseudo-class vs pseudo-element?	State (:hover) vs part of the element (::first-letter)
Four parts of the box model?	Content, padding, border, margin
What does box-sizing: border-box do?	Makes width include padding and border
Order of margin: 10px 20px 30px 40px?	Top, Right, Bottom, Left (clockwise)
How do you centre a block horizontally?	Give it a width, then margin: 0 auto
display: none vs visibility: hidden?	none removes the space; hidden keeps the space
Which overflow value adds scrollbars only if needed?	auto
Default value of position?	static
absolute is positioned relative to what?	Nearest ancestor with position other than static
When does z-index have no effect?	On static elements, or across separate stacking contexts
What does clear: both do?	Pushes the element below all preceding floats
Minimum contrast ratio for body text (AA)?	4.5 : 1

Unit 3 — JavaScript

3.1 JavaScript Introduction

HTML describes structure and CSS describes appearance. Neither can **react**. JavaScript is the behaviour layer: it runs inside the browser, listens for what the user does, changes the page in response, talks to servers, and stores data on the device. A page without JavaScript is a printed poster; with JavaScript it becomes an application.

Role of JavaScript in web development

Area	What JavaScript does
Interactivity	menus, tabs, sliders, accordions, modals, drag and drop
Form validation	checking input before it is sent, with instant feedback
Dynamic content (DOM)	creating, changing and removing elements after the page has loaded
Communication	fetching data from an API with <code>fetch()</code> without reloading the page
Storage	remembering a theme choice or a draft in <code>localStorage</code>
Animation and media	canvas drawing, charts, audio and video control
Beyond the browser	servers (Node.js), mobile apps (React Native), desktop apps (Electron)

Two facts worth remembering for exams. First, JavaScript has **nothing to do with Java** — the name was a 1995 marketing decision. Second, the official standard is **ECMAScript**; "ES6" (2015) is the version that introduced **let**, **const**, arrow functions and template literals, all of which appear throughout this unit.

Internal and external JavaScript

As with CSS, the code can live inside the HTML file or in its own **.js** file.

```
<!-- INTERNAL: a <script> block inside the HTML -->
<body>
  <h1 id="greeting">Hello</h1>

  <script>
    console.log("This script is internal.");
    document.getElementById("greeting").textContent = "Hello, student!";
  </script>
</body>
```

```
<!-- EXTERNAL: link to a separate file (preferred) -->
<body>
  <h1 id="greeting">Hello</h1>

  <script src="js/script.js"></script>
</body>
```

```
// js/script.js - a .js file contains ONLY JavaScript, never <script> tags
console.log("This script is external.");
document.getElementById("greeting").textContent = "Hello, student!";
```

	Internal	External
Location	<script> inside the HTML	separate .js file, linked with src
Reusable across pages	No	Yes
Browser caching	No	Yes
Keeps HTML readable	No	Yes
Best for	tiny demos, quick tests	all real project work

Where to put the <script> tag. Scripts run in order, and a script that runs before the HTML exists cannot find its elements. Two safe options: place the script tag **just before the closing </body> tag**, or place it in <head> with the **defer** attribute — `<script src="js/script.js" defer></script>` — which downloads it in parallel and runs it after the HTML is parsed. A <script> with a src attribute must not also contain code between its tags; the inline code would be ignored.

The browser console

The console is the developer's laboratory. Open DevTools with **F12** (or Ctrl+Shift+I on Windows/Linux, Cmd+Option+I on Mac) and choose the **Console** tab. Three things happen there:

1. Any JavaScript **error** is reported, with the file name and line number. Reading the error message is the fastest debugging skill you can learn.
2. Anything your code sends with `console.log()` is printed.
3. You can type JavaScript directly and see the result immediately — an excellent way to test an idea without editing a file.

Method	Use
console.log(x)	general output; accepts several comma-separated values
console.error(x)	prints in red with a stack trace
console.warn(x)	prints as a yellow warning
console.table(arr)	displays an array of objects as a readable table
console.dir(el)	shows a DOM element as an inspectable object
console.clear()	empties the console

```

let name = "Riya";
let marks = 87;

console.log(name); // Riya
console.log("Name:", name, "Marks:", marks);
console.log(`${name} scored ${marks}%`); // template literal, back-ticks
console.log(typeof marks); // number

console.table([
  { id: 1, name: "Riya", marks: 87 },
  { id: 2, name: "Aman", marks: 74 }
]);

```

console.log() does not change the page — it is purely for the developer.

JavaScript in front-end validation and interaction

Front-end (client-side) validation checks the user's input in the browser before anything is submitted. Its purpose is a **fast, friendly experience**: the user sees "Enter a valid 10-digit mobile number" instantly instead of waiting for a page reload.

```

const form = document.getElementById("registerForm");
const email = document.getElementById("email");
const msg = document.getElementById("emailError");

form.addEventListener("submit", function (event) {
  const pattern = /^[\\w.-]+@[\\w-]+\\. [a-z]{2,}$/i;

  if (!pattern.test(email.value.trim())) {
    event.preventDefault(); // stop the form from submitting
    msg.textContent = "Please enter a valid email address.";
    email.focus();
  }
});

```

A typical validation handler: test the value, block submission, explain the problem.

Client-side validation is never enough. A user can disable JavaScript or send a request directly with a tool. Client-side checks are for **convenience**; the server must validate everything again for **security**. This is a favourite viva question.

3.2 JavaScript Interactions

The browser provides three built-in dialog boxes. They are **modal and blocking** — the page freezes until the user responds — so real projects use custom HTML modals instead. But they are perfect for learning, because they show input and output with one line of code.

Function	Shows	Returns
alert(message)	a message and an OK button	undefined (nothing useful)
prompt(message, default)	a message, a text box, OK and Cancel	the typed text as a string, or null if Cancel
confirm(message)	a message, OK and Cancel	true for OK, false for Cancel

```
// alert – output only
alert("Registration successful!");

// prompt – input; the second argument pre-fills the box
let city = prompt("Which city are you from?", "Anand");

// confirm – a yes/no decision
let sure = confirm("Delete this record permanently?");
if (sure) {
    alert("Record deleted.");
} else {
    alert("Cancelled.");
}
```

Worked example — adding two numbers, and the trap in it

This is the classic first program, and it contains the single most instructive beginner bug in JavaScript.

```
// THE BUG
let a = prompt("Enter first number");    // user types 10 -> "10" (string)
let b = prompt("Enter second number");   // user types 20 -> "20" (string)

alert(a + b);    // shows 1020, not 30!
```

prompt() ALWAYS returns a string. The + operator joins two strings instead of adding them.

The fix is **explicit type conversion** — turn the text into a number before doing arithmetic. Any of these work:

```
let a = Number(prompt("Enter first number"));    // preferred: clear and strict
let b = parseInt(prompt("Enter second number")); // stops at the first non-digit
let c = +prompt("Enter third number");           // unary plus: short but cryptic

let sum = a + b;
alert("The sum is " + sum);    // 30
console.log(typeof a, typeof b); // number number
```

A complete, defensive version that also handles Cancel and non-numeric text:

```
let first = prompt("Enter the first number:");

if (first === null) {                // user pressed Cancel
    alert("Operation cancelled.");
} else {
    let second = prompt("Enter the second number:");
    let n1 = Number(first);
    let n2 = Number(second);

    if (isNaN(n1) || isNaN(n2)) {    // user typed letters
        alert("Please enter numeric values only.");
    } else {
        alert(`${n1} + ${n2} = ${n1 + n2}`);
    }
}
```

Note: use === null to detect Cancel, because an empty box returns "", which is different from null.

Remember these three lines. `prompt()` returns a **string** (or null). `"10" + "20"` is `"1020"`.
`Number("10") + Number("20")` is 30. Almost every exam question on this topic tests exactly that.

3.3 Variables and Data Types

A variable is a named container for a value. Declaring one has two parts: a **keyword** (`var`, `let` or `const`) and a **name**.

```
let  studentName = "Mrugendra"; // can be reassigned later
const COLLEGE    = "CHARUSAT";  // cannot be reassigned
var   oldStyle    = 42;          // legacy – avoid in new code

studentName = "Riya"; // fine
// COLLEGE = "Other"; // TypeError: Assignment to constant variable
```

Naming rules and conventions

- May contain letters, digits, `_` and `$`; **must not start with a digit**.
- Case-sensitive: `marks`, `Marks` and `MARKS` are three different variables.
- Cannot be a reserved word (`let`, `class`, `return`, `function`, `new`, ...).
- Convention: **camelCase** for variables and functions (`totalMarks`, `calculateCgpa`), **UPPER_SNAKE_CASE** for true constants (`MAX_USERS`), **PascalCase** for classes (`StudentRecord`).
- Choose meaning over brevity: `n` tells the reader nothing; `numberOfStudents` tells them everything.

var vs let vs const

	var	let	const
Scope	function	block { }	block { }
Redeclare in same scope	Allowed	Error	Error
Reassign value	Allowed	Allowed	Not allowed
Hoisting	hoisted, value undefined	hoisted but in the *temporal dead zone*	same as let
Must be initialised	No	No	Yes — at declaration
Recommendation	avoid	use when the value changes	default choice

```

if (true) {
  var x = 1;
  let y = 2;
}
console.log(x); // 1      - var leaked out of the block
console.log(y); // ReferenceError - let stayed inside

console.log(a); // undefined - var is hoisted with no value
var a = 5;

console.log(b); // ReferenceError - let is in the temporal dead zone
let b = 5;

```

What const really protects. const freezes the **binding**, not the contents. You cannot point the name at a different value, but if it holds an object or array you may still change what is inside it: `const list = [1,2]; list.push(3);` is legal, while `list = [9]` is not. Practical rule: **use const by default, switch to let only when you genuinely need to reassign, and never use var.**

Data types

JavaScript is **dynamically typed** — a variable's type is decided by the value it currently holds and can change at runtime. There are seven primitive types and one composite type.

Type	Example values	typeof returns	Notes
Number	42, -7, 3.14, 2e3, Infinity, NaN	"number"	one type for integers and decimals
String	"Hi", 'Hi', Hi \${name}	"string"	immutable text; three quote styles
Boolean	true, false	"boolean"	the result of every comparison
Undefined	undefined	"undefined"	declared but no value assigned yet
Null	null	"object"	deliberate "empty" — the typeof result is a famous language bug
BigInt	9007199254740993n	"bigint"	integers beyond the safe number range
Symbol	Symbol("id")	"symbol"	unique keys; rarely needed at this level
Object	{}, [], function() {}, new Date()	"object" / "function"	the only non-primitive type

```

let marks    = 87;           // Number
let name     = "Riya";       // String
let passed   = true;         // Boolean
let address;                 // Undefined (declared, never assigned)
let middle   = null;         // Null      (assigned "nothing" on purpose)

console.log(typeof marks);    // "number"
console.log(typeof address);  // "undefined"
console.log(typeof middle);   // "object"  <-- historical bug, memorise it
console.log(typeof [1,2,3]);  // "object"
console.log(Array.isArray([1,2,3])); // true  <-- correct way to test an array

```

Number, and the special values

```

let a = 10 / 3;           // 3.3333333333333335 (floating point)
let b = 10 / 0;           // Infinity
let c = "abc" * 2;        // NaN ("Not a Number" – but typeof NaN is "number")

console.log(0.1 + 0.2);    // 0.30000000000000004 (binary floats!)
console.log((0.1 + 0.2).toFixed(2)); // "0.30" – toFixed returns a STRING
console.log(isNaN(c));      // true
console.log(Number.isInteger(7)); // true

```

String basics

```

let first = "Mrugendra";
let last  = 'Patel';

let full  = first + " " + last;           // concatenation
let msg   = `Welcome, ${first}!`;        // template literal (back-ticks)

console.log(full.length);                 // 15
console.log(full.toUpperCase());           // MRUGENDRA PATEL
console.log(full.includes("Patel"));      // true
console.log(full.slice(0, 9));             // Mrugendra
console.log(" spaced ".trim());            // "spaced"
console.log("a,b,c".split(","));          // ["a", "b", "c"]

let quote = "She said \"hello\" loudly."; // \" escapes a quote

```

Template literals also allow real line breaks inside the back-ticks — ordinary quotes do not.

Object type — an overview

An object stores several related values together as **key : value** pairs. Where a variable holds one fact, an object holds a whole record.

```

const student = {
  id: 23001,
  name: "Riya Shah",
  branch: "IT",
  cgpa: 8.6,
  isActive: true,
  subjects: ["Web Dev", "DBMS", "OS"], // array value
  address: { city: "Anand", state: "Gujarat" }, // nested object

```

```

    greet: function () {                                // a method
        return `Hello, ${this.name}`;
    }
};

// Reading properties
console.log(student.name);                             // dot notation      -> Riya Shah
console.log(student["branch"]);                         // bracket notation  -> IT
console.log(student.address.city);                     // nested           -> Anand
console.log(student.subjects[0]);                     // array element     -> Web Dev
console.log(student.greet());                          // calling a method

// Adding, updating, removing
student.semester = 5;
student.cgpa = 8.8;
delete student.isActive;

console.log(Object.keys(student));                     // list of property names
console.log(student.email);                           // undefined – missing key, not an error

```

- Use **dot notation** normally. Use **bracket notation** when the key is stored in a variable or is not a valid identifier: `student[field], data["first name"]`.
- An **array** is a specialised object whose keys are numeric indexes starting at 0: `let marks = [88, 74, 91]`; with `marks.length, marks.push(), marks[0]`.
- Objects are **reference types**. `let b = a`; copies the reference, not the data — changing `b.name` also changes `a.name`. Primitives are copied by value.
- Objects written in this literal form are the basis of **JSON**, the format used to exchange data with servers.

3.4 Type Conversion, Operators and Math

Explicit conversion

Goal	Function	Examples
To string	<code>String(x)</code> or <code>x.toString()</code>	<code>String(25) -> "25"</code> ; <code>true.toString() -> "true"</code>
To number	<code>Number(x)</code>	<code>Number("25") -> 25</code> ; <code>Number("25a") -> NaN</code> ; <code>Number("") -> 0</code>
To integer	<code>parseInt(x)</code>	<code>parseInt("25px") -> 25</code> ; <code>parseInt("px25") -> NaN</code>
To decimal	<code>parseFloat(x)</code>	<code>parseFloat("3.75kg") -> 3.75</code>
To number (short)	<code>+x</code>	<code>+"25" -> 25</code>
To boolean	<code>Boolean(x)</code>	<code>Boolean("") -> false</code> ; <code>Boolean("0") -> true</code>

The difference between `Number()` and `parseInt()` is examined often: `Number()` is **strict** — the whole string must be a valid number, otherwise `NaN`. `parseInt()` is **tolerant** — it reads digits from the start and stops at the first character that is not part of a number.

```

Number("42")      // 42
Number("42abc")   // NaN
Number("")        // 0      <-- surprising but true
Number(" 42 ")    // 42    (whitespace is trimmed)
Number(true)      // 1
Number(null)      // 0
Number(undefined) // NaN

parseInt("42abc") // 42
parseInt("3.9")   // 3      (truncates, does not round)
parseFloat("3.9") // 3.9
parseInt("ff", 16) // 255   (second argument = radix)

```

Truthy and falsy

In a boolean context every value is either **truthy** or **falsy**. There are exactly **eight falsy values** — memorise this short list, and everything else is **truthy**.

```

// FALSY: false  0  -0  0n  ""  null  undefined  NaN
// TRUTHY: everything else, including these surprises:

Boolean("0")      // true   - a non-empty string
Boolean("false")  // true   - still a non-empty string
Boolean([])       // true   - an empty array is an object
Boolean({})       // true   - an empty object is an object
Boolean(-5)       // true   - only 0 is falsy

let username = "";
if (!username) { console.log("Name is required"); } // runs

```

Implicit conversion (coercion)

JavaScript converts types automatically when an operator receives a type it did not expect. The rule to hold on to: **the + operator prefers strings; every other arithmetic operator prefers numbers**.

Expression	Result	Why
"5" + 2	"52"	+ with a string concatenates
"5" - 2	3	- has no string meaning, so both become numbers
"5" * "2"	10	both converted to numbers
5 + true	6	true becomes 1
5 + null	5	null becomes 0
5 + undefined	NaN	undefined becomes NaN
"5" == 5	true	== converts before comparing
"5" === 5	false	=== compares value and type
null == undefined	true	special case in the == rules
null === undefined	false	different types
NaN == NaN	false	NaN is not equal to anything, including itself

Always use === and !==. The loose operators == and != perform hidden conversions and produce results such as "" == 0 being true and " \t" == 0 being true. Strict comparison has no surprises. Use == only for the deliberate idiom x == null, which tests for null **or** undefined in one go.

Operators

Category	Operators	Note
Arithmetic	+ - * / % **	% is remainder; is exponent (2 ¹⁰ = 1024)
Increment / decrement	++ --	prefix ++x increments then returns; postfix x++ returns then increments
Assignment	= += -= *= /= %= **=	x += 5 is short for x = x + 5
Comparison	== === != !== > < >= <=	always produce a boolean
Logical	&& !	AND, OR, NOT — with short-circuit evaluation
Ternary	condition ? a : b	the only operator taking three operands
Nullish coalescing	??	returns the right side only if the left is null/undefined
Type / other	typeof instanceof	inspect a value's type

```
let a = 17, b = 5;
console.log(a + b, a - b, a * b); // 22 12 85
console.log(a / b);               // 3.4 (no integer division)
console.log(a % b);               // 2 (remainder — used for even/odd)
console.log(a ** 2);              // 289

let x = 5;
console.log(x++); // prints 5, then x becomes 6
console.log(++x); // x becomes 7, prints 7

let total = 100;
total += 25; // 125
total *= 2;  // 250
```

Logical operators and short-circuiting

&& returns the **first falsy** operand, or the last one if all are truthy. **||** returns the **first truthy** operand, or the last one if all are falsy. They do not necessarily return **true/false** — this is what makes the default-value idiom work.

```

let age = 20, hasId = true;
if (age >= 18 && hasId) { console.log("Entry allowed"); }
if (age < 18 || !hasId) { console.log("Entry denied"); }

// Short-circuit: the right side is never evaluated if the answer is known
let userName = inputName || "Guest";    // fallback when inputName is falsy
let count     = userCount ?? 0;         // fallback ONLY for null/undefined

// Careful: 0 is falsy, so || replaces a legitimate zero. ?? does not.
let score = 0;
console.log(score || 10);    // 10 <-- probably a bug
console.log(score ?? 10);    // 0  <-- correct

```

The ternary operator

```

// condition ? valueIfTrue : valueIfFalse
let marks = 72;
let result = marks >= 40 ? "Pass" : "Fail";

console.log(`Status: ${marks >= 40 ? "Pass" : "Fail"}`);

// Chained (readable only when kept short)
let grade = marks >= 80 ? "A" : marks >= 60 ? "B" : marks >= 40 ? "C" : "F";

```

A ternary is an expression, so it can sit inside a string or be assigned. An if statement cannot.

Operator precedence — a short reference

Order	Operators
1 (highest)	() grouping
2	++ -- ! typeof unary - unary +
3	
4	* / %
5	+ -
6	< <= > >=
7	== != === !==
8	&&
9	??
10	? : ternary
11 (lowest)	= += -= and other assignments

When in doubt, add parentheses. $2 + 3 * 4$ is 14, but $(2 + 3) * 4$ is 20, and the second version is clearer to a reader even when the brackets are unnecessary.

The Math object

Method	Result	Example
Math.round(x)	nearest integer (.5 rounds up)	Math.round(4.5) -> 5

Method	Result	Example
Math.floor(x)	rounds down	Math.floor(4.9) -> 4
Math.ceil(x)	rounds up	Math.ceil(4.1) -> 5
Math.trunc(x)	drops the decimal part	Math.trunc(-4.9) -> -4
Math.abs(x)	absolute value	Math.abs(-7) -> 7
Math.pow(x, y)	x to the power y	Math.pow(2, 8) -> 256
Math.sqrt(x)	square root	Math.sqrt(81) -> 9
Math.min(...) / Math.max(...)	smallest / largest argument	Math.max(3, 9, 5) -> 9
Math.random()	random decimal, $0 \leq r < 1$	0.5731...
Math.PI	3.14159...	constant, not a function

```
// Random integer between min and max inclusive – memorise this recipe
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 6));    // a dice roll: 1 to 6

// Rounding to 2 decimal places
let price = 249.6789;
console.log(Math.round(price * 100) / 100); // 249.68 (a number)
console.log(price.toFixed(2));              // "249.68" (a STRING)

// Largest value in an array
let marks = [78, 92, 65, 88];
console.log(Math.max(...marks));            // 92 (spread operator)
```

3.5 Conditional Statements and Loops

if / else if / else

```
let marks = 76;

if (marks >= 90) {
  console.log("Grade: A+");
} else if (marks >= 80) {
  console.log("Grade: A");
} else if (marks >= 70) {
  console.log("Grade: B");    // <-- this one runs
} else if (marks >= 40) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

Conditions are tested top to bottom; the **FIRST** true branch runs and the rest are skipped. Order the conditions carefully — putting `marks >= 40` first would give everyone a C.

- The braces `{ }` are optional for a single statement but **always write them** — omitting them causes real bugs when a second line is added later.

- `if (x = 5)` uses the assignment operator: it assigns 5 and then evaluates as `truthy`, so the branch always runs. You meant `if (x === 5)`.
- Nested `if` statements are legal but deep nesting hurts readability — prefer `else if` chains or an early `return` inside a function.

switch

When one value is compared against many fixed options, `switch` reads better than a long `else if` chain. It compares using **strict equality** (`===`), so `case "1"` will not match the number `1`.

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday"); // matches
    break;
  default:
    console.log("Invalid day");
}
```

Forgetting break causes fall-through. Without `break`, execution continues into the next case regardless of whether it matches. That is usually a bug — but it can be used deliberately to group cases, as below. `default` handles anything unmatched and may be placed anywhere, though the end is conventional.

```
let day = "SUN";

switch (day) {
  case "SAT":
  case "SUN": // deliberate grouped fall-through
    console.log("Weekend - holiday");
    break;
  case "MON": case "TUE": case "WED": case "THU": case "FRI":
    console.log("Working day");
    break;
  default:
    console.log("Unknown day code");
}
```

Loops

A loop repeats a block of statements. Choose the one that matches what you know before you start.

Loop	Use when	Runs at least once?
for	the number of repetitions is known	No
while	you repeat until a condition changes	No

Loop	Use when	Runs at least once?
do...while	the body must run before the first test	Yes
for...of	iterating the values of an array or string	No
for...in	iterating the keys of an object	No

```
// for – three parts separated by semicolons
for (let i = 1; i <= 5; i++) {
  //   ^init      ^condition ^update
  console.log("Iteration " + i);
}

// while – the condition is tested BEFORE each pass
let n = 1;
while (n <= 5) {
  console.log(n);
  n++;           // forget this and the loop never ends
}

// do...while – the body runs, THEN the condition is tested
let choice;
do {
  choice = prompt("Enter a number between 1 and 10:");
} while (Number(choice) < 1 || Number(choice) > 10);
```

```
const marks = [88, 74, 91, 65];

for (let i = 0; i < marks.length; i++) console.log(marks[i]); // index-based
for (const m of marks)                 console.log(m);         // values

const student = { name: "Riya", branch: "IT", cgpa: 8.6 };
for (const key in student) console.log(key + " = " + student[key]);
```

for...of gives values, for...in gives keys. Using for...in on an array is a common mistake — it yields index strings, not values.

break and continue

Keyword	Effect
break	exits the loop (or switch) immediately
continue	skips the rest of this pass and starts the next one

```
// break – stop as soon as the target is found
const ids = [101, 102, 103, 104];
for (let i = 0; i < ids.length; i++) {
  if (ids[i] === 103) {
    console.log("Found at index " + i);
    break; // no point checking the rest
  }
}

// continue – skip the unwanted items
for (let i = 1; i <= 10; i++) {
  if (i % 2 !== 0) continue; // skip odd numbers
  console.log(i);           // 2 4 6 8 10
}
```

Infinite loops. A while loop whose condition never becomes false will freeze the browser tab. The usual causes are forgetting the update statement (n++), or updating in the wrong direction. In a for loop, never modify the counter inside the body as well as in the update expression.

Worked examples

```
// 1. Sum of the first n natural numbers
let n = 10, sum = 0;
for (let i = 1; i <= n; i++) { sum += i; }
console.log("Sum =", sum); // 55

// 2. Multiplication table of 7
for (let i = 1; i <= 10; i++) {
  console.log(`7 x ${i} = ${7 * i}`);
}

// 3. FizzBuzz – the classic interview warm-up
for (let i = 1; i <= 15; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}

// 4. Nested loops – a star triangle
let out = "";
for (let i = 1; i <= 5; i++) {
  let row = "";
  for (let j = 1; j <= i; j++) { row += "* "; }
  out += row + "\n";
}
console.log(out);

// 5. Highest mark and average
const marks = [88, 74, 91, 65, 79];
let highest = marks[0], total = 0;
for (const m of marks) {
  if (m > highest) highest = m;
  total += m;
}
console.log("Highest:", highest, "Average:", (total / marks.length).toFixed(2));
```

3.6 Functions

A function is a named, reusable block of code that runs only when it is called. Functions exist to avoid repetition, to give a name to an idea, and to let you test one piece of logic in isolation. The guiding principle is **one function, one job**.

Function declaration

```
// Definition
function greet(name) {           // "name" is a PARAMETER
  return "Hello, " + name + "!";
}

// Call (invocation)
let message = greet("Riya");      // "Riya" is an ARGUMENT
console.log(message);             // Hello, Riya!
```

A declaration is **hoisted**: the whole function is moved to the top of its scope, so it can be called on a line above where it is written.

```
console.log(square(4));           // 16 – works, because declarations hoist

function square(x) { return x * x; }
```

Function expression

Here the function is a **value** stored in a variable. It is **not** hoisted — the variable must be assigned before it can be called.

```
const square = function (x) {    // anonymous function expression
  return x * x;
};
console.log(square(4));           // 16

// console.log(cube(3));          // TypeError: cube is not a function
const cube = function (x) { return x * x * x; };
```

Arrow function (ES6)

A shorter syntax for function expressions. The **function** keyword is dropped and **=>** is placed between the parameters and the body.

```
// Full form
const square = (x) => {
  return x * x;
};

// One parameter -> brackets optional
const square = x => { return x * x; };

// Single expression -> braces and return can both go (implicit return)
const square = x => x * x;

// No parameters -> empty brackets are required
const now = () => new Date().toLocaleTimeString();

// Several parameters
const add = (a, b) => a + b;

// Returning an object literal -> wrap it in round brackets
const makeUser = name => ({ name: name, active: true });

// Very common use: as a callback
const marks = [88, 74, 91];
const passed = marks.filter(m => m >= 80);      // [88, 91]
const doubled = marks.map(m => m * 2);          // [176, 148, 182]

button.addEventListener("click", () => alert("Clicked!"));
```

	Declaration	Expression	Arrow
Syntax	function f() {}	const f = function () {}	const f = () => {}
Hoisted	Yes	No	No
Implicit return	No	No	Yes, for a single expression
Own this	Yes	Yes	No — inherits from the enclosing scope
arguments object	Yes	Yes	No
Usable as a method / constructor	Yes	Yes	Not as a constructor
Best for	named top-level utilities	assigning conditionally	short callbacks

Parameters and return values

```
// Default parameter – used when the argument is missing or undefined
function greet(name = "Guest", greeting = "Hello") {
  return `${greeting}, ${name}!`;
}
console.log(greet());           // Hello, Guest!
console.log(greet("Riya", "Hi")); // Hi, Riya!

// Missing argument without a default becomes undefined
function add(a, b) { return a + b; }
console.log(add(5));           // NaN (5 + undefined)

// Rest parameter – collects any number of arguments into an array
```

```
function total(...numbers) {
  let sum = 0;
  for (const n of numbers) sum += n;
  return sum;
}
console.log(total(10, 20, 30, 40)); // 100

// return ends the function immediately
function checkAge(age) {
  if (age < 0) return "Invalid"; // early return – no else needed
  if (age < 18) return "Minor";
  return "Adult";
}

// A function with no return statement gives back undefined
function log(msg) { console.log(msg); }
console.log(log("hi")); // hi then undefined
```

One value only. A function returns a single value. To send several results back, return an **object** or an **array**: `return { min: a, max: b };` and read it with `const { min, max } = getRange(marks);` (destructuring).

Scope

Scope is the region of code in which a variable is visible. JavaScript has three levels.

Scope	Created by	Visible to
Global	declaring outside every function and block	the entire script — and everything else on the page
Function (local)	declaring inside a function	that function and any function nested in it
Block	declaring with <code>let/const</code> inside <code>{ }</code>	that block only (if, for, while, plain braces)

```
let globalVar = "I am global";

function demo() {
  let localVar = "I am local";
  console.log(globalVar); // OK – inner code can see outer variables

  if (true) {
    let blockVar = "I am block-scoped";
    var funcVar = "var ignores blocks";
    console.log(localVar); // OK
  }

  console.log(funcVar); // OK – var is function-scoped
  // console.log(blockVar); // ReferenceError – let is block-scoped
}

demo();
// console.log(localVar); // ReferenceError – not visible outside
```

- **Scope chain:** when a name is used, JavaScript looks in the current scope, then the enclosing scope, and outward until it reaches global. If it is never found, a **ReferenceError** is thrown.
- **Shadowing:** an inner variable with the same name hides the outer one for the duration of that scope. Legal, but confusing — pick different names.
- **Avoid globals.** They can be overwritten by any other script on the page. Keep variables in the smallest scope that works.
- **Closure (preview):** an inner function keeps access to its outer function's variables even after the outer function has finished. This is what makes event handlers and callbacks remember their surrounding data.

```
// Closure in action – each call gets its own private counter
function makeCounter() {
  let count = 0;                // private to this call
  return function () {
    count++;
    return count;
  };
}
const next = makeCounter();
console.log(next(), next(), next()); // 1 2 3
```

Two complete examples

```
// 1. Reusable validation helpers for a registration form
const isValidEmail = email => /^[w.-]+@[w-]+\.[a-z]{2,}$/i.test(email.trim());
const isValidMobile = mobile => /^[6-9]\d{9}$/i.test(mobile.trim());

function validateForm(email, mobile, password) {
  const errors = [];

  if (!isValidEmail(email)) errors.push("Invalid email address.");
  if (!isValidMobile(mobile)) errors.push("Mobile must be 10 digits.");
  if (password.length < 8) errors.push("Password too short.");

  return errors;                // empty array means the form is valid
}

const problems = validateForm("riya@example.com", "9876543210", "abc");
console.log(problems.length === 0 ? "Valid" : problems.join(" "));
```

```
// 2. Grade calculator combining conditionals, loops and functions
function averageOf(marks) {
  if (marks.length === 0) return 0;
  let total = 0;
  for (const m of marks) total += m;
  return total / marks.length;
}

function gradeFor(average) {
  if (average >= 90) return "A+";
  if (average >= 80) return "A";
  if (average >= 70) return "B";
  if (average >= 40) return "C";
  return "F";
}

function report(name, marks) {
  const avg = averageOf(marks);
  return `${name}: average ${avg.toFixed(2)}, grade ${gradeFor(avg)}`;
}

console.log(report("Riya", [88, 74, 91, 65, 79]));
// Riya: average 79.40, grade B
```

Three small functions, each with one job, combined into a result — this is how real code is structured.

3.7 Unit 3 — Quick Revision

Question	One-line answer
Where does JavaScript run?	In the browser's JavaScript engine (also on servers via Node.js)
Two ways to include JS?	Internal <code><script></code> block, or external .js file via <code>src</code>
Why use <code>defer</code> ?	Downloads in parallel and runs after the HTML is parsed
What does <code>prompt()</code> return?	Always a string, or null if Cancel is pressed
What does <code>confirm()</code> return?	A boolean — true for OK, false for Cancel
Why is <code>"10" + "20"</code> equal to <code>"1020"</code> ?	<code>+</code> with strings concatenates; convert with <code>Number()</code> first
Difference between <code>let</code> and <code>var</code> ?	<code>let</code> is block-scoped and cannot be redeclared; <code>var</code> is function-scoped
Can a <code>const</code> object be changed?	Its properties can; the variable cannot be reassigned
<code>typeof null</code> returns?	<code>"object"</code> — a well-known language bug
How many falsy values are there?	Eight: <code>false</code> , <code>0</code> , <code>-0</code> , <code>0n</code> , <code>""</code> , <code>null</code> , <code>undefined</code> , <code>NaN</code>
Difference between <code>==</code> and <code>===</code> ?	<code>==</code> converts types first; <code>===</code> compares value and type
<code>Number()</code> vs <code>parseInt()</code> ?	<code>Number()</code> is strict (<code>NaN</code> on any junk); <code>parseInt()</code> reads leading digits
Which loop always runs once?	<code>do...while</code>
What does <code>break</code> do in a loop?	Exits the loop immediately
What does <code>continue</code> do?	Skips to the next iteration
Why does <code>switch</code> need <code>break</code> ?	Otherwise execution falls through into the following cases
Which comparison does <code>switch</code> use?	Strict equality (<code>===</code>)
Are function declarations hoisted?	Yes; function expressions and arrow functions are not
What does a function with no <code>return</code> give back?	<code>undefined</code>
What is scope?	The region of code where a variable is visible
Random integer 1–6?	<code>Math.floor(Math.random() * 6) + 1</code>

Practice problems

Attempt each of these in the browser console before checking the notes again.

1. Ask the user for two numbers with `prompt()` and display their sum, difference, product and quotient in a single `alert()`.
2. Ask for a number and print whether it is positive, negative or zero using `if / else if / else`.
3. Ask for a marks value and print the grade using a `switch` on `Math.floor(marks / 10)`.
4. Print all even numbers from 1 to 50 using a `for` loop and `continue`.
5. Given `const marks = [88, 74, 91, 65, 79]`, find the highest, the lowest and the average without using `Math.max`.
6. Write a function `isPrime(n)` that returns `true` or `false`, then print all primes below 50.

7. Write an arrow function `celsiusToFahrenheit(c)` and print a conversion table from 0 to 100 in steps of 10.
8. Create a `student` object with at least five properties including one array and one method, then print each key and value using `for...in`.
9. Write `countVowels(str)` that returns how many vowels a string contains.
10. Print the following pattern using nested loops: a right-aligned triangle of 5 rows of stars.

Section 3 — Predict the Output

Write down what each snippet prints **before** reading the answers. If you are unsure, reason it out on paper first, then verify in the browser console — the gap between your guess and the real result is where the learning is.

Snippet 1

```
console.log("5" + 3);
console.log("5" - 3);
console.log("5" * "2");
console.log(5 + true);
console.log(5 + null);
console.log(5 + undefined);
```

Snippet 2

```
let x = 10;
console.log(x++);
console.log(x);
console.log(++x);
console.log(x--);
console.log(x);
```

Snippet 3

```
console.log(typeof 42);
console.log(typeof "42");
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof null);
console.log(typeof [1, 2]);
console.log(typeof function () {});
```

Snippet 4

```
if (true) {
  var a = 1;
  let b = 2;
}
console.log(a);
try { console.log(b); } catch (e) { console.log("Error: " + e.name); }
```

Snippet 5

```
let day = 2;
switch (day) {
  case 1: console.log("Mon");
  case 2: console.log("Tue");
  case 3: console.log("Wed");
  default: console.log("Other");
}
```

Snippet 6

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  if (i === 5) break;
  console.log(i);
}
```

Snippet 7

```
console.log(square(4));
function square(x) { return x * x; }

try { console.log(cube(2)); } catch (e) { console.log("Error: " + e.name); }
const cube = x => x * x * x;
```

Snippet 8

```
function greet(name = "Guest") { return "Hi, " + name; }
console.log(greet());
console.log(greet(undefined));
console.log(greet(null));
console.log(greet(""));
```

Snippet 9

```
let score = 0;
console.log(score || 10);
console.log(score ?? 10);
console.log("" || "default");
console.log(Boolean("0"), Boolean([]), Boolean(""));
```

Snippet 10

```
const s = { name: "Riya", marks: [80, 90] };
const t = s;
t.name = "Aman";
console.log(s.name);

let p = 5;
let q = p;
q = 9;
console.log(p);
```

Snippet 11

```
console.log(Math.round(4.5), Math.round(-4.5));
console.log(Math.floor(4.9), Math.ceil(4.1));
console.log(Math.trunc(-4.9), Math.floor(-4.9));
console.log((2.567).toFixed(2), typeof (2.567).toFixed(2));
```

Snippet 12 (CSS)

```
<p id="a" class="b">Text</p>

p      { color: red;   }
.b     { color: green; }
p.b    { color: blue;  }
#a     { color: orange;}
p#a.b  { color: purple;}
```

Which colour is the text, and why?

Answers — Section 3

#	Output	Reason
1	"53" · 2 · 10 · 6 · 5 · NaN	+ concatenates with a string; -, * convert to numbers; true is 1; null is 0; undefined becomes NaN
2	10 · 11 · 12 · 12 · 11	Postfix returns the old value then changes it; prefix changes it first
3	number · string · boolean · undefined · object · object · function	typeof null is "object" (a language bug); arrays are objects; use Array.isArray to test an array
4	1 then Error: ReferenceError	var is function-scoped and leaks out of the block; let is block-scoped
5	Tue · Wed · Other	No break statements, so execution falls through every following case
6	1 · 2 · 4	continue skips 3; the loop stops at 5 before the log runs
7	16 then Error: ReferenceError	Function declarations are hoisted; const arrow functions are not
8	Hi, Guest · Hi, Guest · Hi, null · Hi,	A default applies only for undefined — null and "" are passed through as real values
9	10 · 0 · "default" · true false false	treats 0 as falsy; ?? only replaces null/undefined; "0" and [] are truthy
10	Aman then 5	Objects are copied by reference; primitives are copied by value
11	5 -4 · 4 5 · -4 -5 · "2.57" string	Math.round breaks .5 ties upward (towards +Infinity); trunc drops the decimal, floor goes down; toFixed returns a string
12	purple	p#a.b scores 1-1-1, higher than #a (1-0-0), p.b (0-1-1), .b (0-1-0) and p (0-0-1)

Section 4 — Find the Bug

Each snippet contains exactly one defect. Identify it and write the corrected line before turning to the fixes.

Bug 1

```
let a = prompt("First number");
let b = prompt("Second number");
alert("Sum = " + (a + b));
```

Bug 2

```
let marks = 45;
if (marks = 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Bug 3

```
let n = 1;
while (n <= 5) {
  console.log(n);
}
```

Bug 4

```
const marks = [88, 74, 91];
for (const m in marks) {
  console.log(m * 2);
}
```

Bug 5

```
const total = 0;
for (let i = 1; i <= 5; i++) {
  total += i;
}
console.log(total);
```

Bug 6 (CSS)

```
.card {
  border-width: 3px;
  border-color: navy;
}
```

Bug 7 (CSS)

```
.badge {  
  position: absolute;  
  top: 10px;  
  right: 10px;  
}  
.card {  
  padding: 20px;  
  border: 1px solid #ccc;  
}
```

The badge appears in the corner of the page instead of the corner of the card.

Bug 8 (CSS)

```
.wrapper {  
  margin: 0 auto;  
  background: #eee;  
}
```

The wrapper is not centred.

Bug 9

```
document.getElementById("btn").addEventListener("click", showAlert);  
  
<!-- ...and in the HTML, further down the page: -->  
<button id="btn">Click me</button>
```

The console reports: Cannot read properties of null.

Bug 10

```
function checkAge(age) {  
  if (age >= 18) {  
    "Adult";  
  } else {  
    "Minor";  
  }  
}  
console.log(checkAge(21));
```

Fixes — Section 4

#	The defect	The fix
1	prompt returns strings, so + concatenates: "1020" instead of 30	Convert first: let a = Number(prompt(...)); or use +prompt(...)
2	Assignment (=) inside the condition, not comparison; marks becomes 40, which is truthy, so Pass always prints	Use if (marks >= 40) — and use === for equality
3	The counter is never updated, so the condition stays true forever	Add n++; as the last statement inside the loop
4	for...in over an array yields index strings "0", "1", "2", so m * 2 gives 0, 2, 4	Use for (const m of marks) to iterate the values
5	total is declared const, so += throws TypeError: Assignment to constant variable	Declare it with let, since the value changes
6	border-style defaults to none, so nothing is drawn	Add border-style: solid; or use the shorthand border: 3px solid navy;
7	The card is position: static, so the absolute badge falls back to the page	Add position: relative; to .card
8	margin: 0 auto needs an explicit width to have leftover space to split	Add width: 1100px; or max-width: 1100px;
9	The script runs before the button exists in the DOM	Move the script just before </body>, or add the defer attribute to the script tag
10	The strings are evaluated and discarded; there is no return, so the function gives undefined	Write return "Adult"; and return "Minor";